Well-Architected Review Rules

What the Project Catalyst pipeline checks in a Statement of Work before the architecture is drawn, exactly as implemented.

Status	Reference of the implemented behaviour. Generated from the source code; every example below was executed against it when this file was built.
Generated	2026-10-27
Rule tests	29 passed (sow-generator-agent/deployment/agent/tests/test_wa_review.py)
What it is	Ingram's own checklist across the six Well-Architected pillars, applied to the text of a SOW.
What it is not	Not the AWS Well-Architected Tool, not the AWS Well-Architected Agent, and not a review of a deployed workload or of the diagram.

1. In one page

Where it runs. Stage 2 of the pipeline: after the SOW exists and before the architecture diagram, the cost estimate and the price are produced. The reason is cost of change: a weak service decision found here costs one SOW edit; found after the diagram and the price it costs a loop through all three stages.

How it decides. The SOW agent applies a fixed list of rules to the SOW text. Nine rules look for a commitment the SOW must make (backup and recovery, encryption, observability and so on); four rules look for something the SOW says that contradicts a pillar (an undecided compute choice, a single-Availability-Zone scope, an unsupported resiliency claim, a missing out-of-scope list). Each finding carries a pillar, a severity, the text it fired on and a suggested fix.

What happens next. Any **high** finding stops the run before the diagram is drawn. A person either edits the SOW and starts again, or chooses *Continue anyway*, which is logged. Medium and low findings are shown and never stop the run.

Deployed setting. The optional model review of the SOW (Section 5) is **off** in the deployed pipeline (`WaVerify=false`). With it off, the result is decided by the rules alone.

How to read a pass. A pass means the SOW mentions each topic. It does not mean the commitment is adequate (Section 7 shows a sentence that passes while promising no backups).

Pillar	Rules	Rule ids
Operational Excellence	4	wa_operations_observability, wa_operations_handover, wa_operations_compute_decision_open, wa_operations_missing_out_of_scope
Security	2	wa_security_encryption, wa_security_identity
Reliability	4	wa_reliability_backup_recovery, wa_reliability_availability_target, wa_reliability_single_az, wa_reliability_overclaimed_resiliency
Performance Efficiency	1	wa_performance_targets
Cost Optimization	1	wa_cost_controls
Sustainability	1	wa_sustainability_efficiency

2. How a run reaches the review

Stage	Agent	What it does
1. SOW draft	SOW	Drafts the SOW from intake and discovery. Skipped if a SOW is supplied.
2. Well-Architected review	SOW	**Applies the rules in this document to the SOW. A high finding blocks.** Also reads the in-scope and out-of-scope lists out of the SOW.
3. Architecture	Architecture	Designs from the SOW and the scope lists. Defaults to two Availability Zones (Section 6).
4. Cost estimate	Cost	Prices the design in the AWS Pricing Calculator. The link is written back into the SOW's Pricing section.
5. Combined review	SOW	Re-checks the finished SOW against the architecture and cost (Section 8).

Severity and the gate. `can_proceed` is true only when the count of high findings is zero. The pipeline stops on a high finding unless the run was started with `stop_on_blocked=false` (the default is stop). Findings are ordered high, then medium, then low, then by pillar order: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

Profiles. The template selects the profile. A template in the `cva` category is an assessment: it installs a collector and writes a report and deploys nothing, so the build-oriented rules would be false blockers. Skipped for assessments: all 9 commitment rules and `wa_operations_compute_decision_open`. Still applied: single-AZ scope, unsupported resiliency claim, and missing out-of-scope list. Every other template uses the full set.

3. Commitment rules (the SOW must say something)

Each rule fires when **none** of its patterns matches anywhere in the SOW (case-insensitive). The rule is satisfied by a single match. The evidence shown is the fixed sentence "The SOW contains no wording on this topic."

wa_reliability_backup_recovery	Reliability | **HIGH** | commitment rule
Finding title	**No backup or recovery commitment**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bRPO\b`
	`\bRTO\b`
	`\bback-?ups?\b`
	`\bsnapshots?\b`
	`point-in-time recovery`
	`\bdisaster recovery\b`
	`\brestore\b`

Finding says	The SOW does not state how the workload recovers from data loss, so RPO and RTO are undefined and the delivery team has no target to build to.
Suggested fix	State the backup mechanism and retention, and give RPO and RTO as outcomes of restoring from backup.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Automated backups with 7-day retention. RPO 15 minutes, RTO 2 hours.
AWS reference	REL09-BP01: Identify and back up all data that needs to be backed up, or reproduce the data from sources https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_backing_up_data_identified_backups_data.html

wa_reliability_availability_target	Reliability | **MEDIUM** | commitment rule
Finding title	**No availability target**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bavailability\b`
	`\buptime\b`
	`\bSLA\b`
	`\bhigh[- ]availability\b`
	`\b99\.\d+\s*%`
	`\bresilien\w+`

Finding says	No availability or uptime target is stated, so there is nothing to design the redundancy level against and nothing to hold delivery to.
Suggested fix	State the availability target (for example 99.9% monthly) or say explicitly that the pilot carries no availability commitment.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Availability target of 99.9% monthly.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

wa_security_encryption	Security | **HIGH** | commitment rule
Finding title	**No encryption commitment**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bencrypt\w*`
	`\bKMS\b`
	`\bTLS\b`
	`\bHTTPS\b`
	`\bat rest\b`
	`\bin transit\b`

Finding says	The SOW does not commit to encrypting data at rest or in transit, which is a baseline Security pillar expectation and a common customer audit question.
Suggested fix	State encryption at rest (KMS) and in transit (TLS), and name the key management approach.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Data is encrypted at rest with AWS KMS and in transit with TLS.
AWS reference	SEC08-BP02: Enforce encryption at rest https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_protect_data_rest_encrypt.html SEC09-BP02: Enforce encryption in transit https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_protect_data_transit_encrypt.html

wa_security_identity	Security | **MEDIUM** | commitment rule
Finding title	**No identity or access-control approach**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bIAM\b`
	`least privilege`
	`\bRBAC\b`
	`\bSSO\b`
	`\bCognito\b`
	`\bEntra\b`
	`\bSAML\b`
	`\bOIDC\b`
	`\bauthenticat\w+`
	`\bauthoris\w+`
	`\bauthoriz\w+`
	`access control`

Finding says	The SOW does not say how access is granted or restricted, so least privilege cannot be verified and the customer cannot review who will hold which permissions.
Suggested fix	Name the identity provider and sign-in model, and commit to least-privilege IAM roles.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Least-privilege IAM roles; sign-in through Microsoft Entra ID (SAML).
AWS reference	SEC03-BP02: Grant least privilege access https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_permissions_least_privileges.html

wa_operations_observability	Operational Excellence | **HIGH** | commitment rule
Finding title	**No observability commitment**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bmonitor\w*`
	`\bobservabilit\w+`
	`\blogg?ing\b`
	`\bCloudWatch\b`
	`\balarms?\b`
	`\balert\w*`
	`\bdashboards?\b`
	`\btracing\b`
	`\bX-Ray\b`

Finding says	The SOW does not commit to logging, metrics or alerting, so the workload would go live with no way to detect or diagnose failure.

Suggested fix	State what is logged, which metrics and alarms exist, and who receives alerts.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Amazon CloudWatch logs, metrics and alarms go to the on-call rota.
AWS reference	OPS08-BP01: Analyze workload metrics (workload observability) https://docs.aws.amazon.com/wellarchitected/latest/framework/ops_workload_observability_analyze_workload_metrics.html

wa_operations_handover	Operational Excellence | **MEDIUM** | commitment rule
Finding title	**No runbook or handover commitment**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\brunbook\b`
	`\bhand-?over\b`
	`\bknowledge transfer\b`
	`\bsupport model\b`
	`\boperational documentation\b`
	`\btraining\b`
	`\bdocumentation\b`

Finding says	No operational handover is described, so it is unclear who runs the workload after go-live or what documentation they receive.
Suggested fix	Add a runbook or operational handover deliverable and name the team that takes ownership.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** A runbook and operational handover to the customer's platform team.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

wa_performance_targets	Performance Efficiency | **MEDIUM** | commitment rule
Finding title	**No performance target**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\blatenc\w+`
	`\bthroughput\b`
	`\brequests? per\b`
	`\btransactions? per\b`
	`\bconcurren\w+`
	`\bresponse time\b`
	`\bp9[59]\b`
	`\bTPS\b`
	`\bQPS\b`

Finding says	The SOW states no latency, throughput or volume target, so the services cannot be sized and the cost estimate rests on guesses.
Suggested fix	State the expected volumes and a latency or throughput target for the main flow.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Expected load is 40 concurrent users with a p95 latency target of 800 ms.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

wa_cost_controls	Cost Optimization | **LOW** | commitment rule
Finding title	**No cost-control approach**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`right[- ]siz\w+`
	`\bSavings Plans?\b`
	`\bReserved Instances?\b`
	`cost allocation`
	`\btagging\b`
	`\bbudgets?\b`
	`\bcost alarm\w*`
	`\bauto[- ]?scal\w+`
	`\bserverless\b`

Finding says	The SOW does not mention how running cost is kept under control, so there is no stated owner or mechanism for cost once the workload is live.
Suggested fix	Mention right-sizing, tagging for cost allocation, budgets or alarms, and any Savings Plan assumption.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Right-sizing review at go-live and a monthly budget alarm.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

wa_sustainability_efficiency	Sustainability | **LOW** | commitment rule
Finding title	**No efficiency or lifecycle commitment**
Fires when	No pattern below matches anywhere in the SOW.

Detects (regex)	`\bGraviton\b`
	`\barm64\b`
	`lifecycle polic\w+`
	`\bstorage class\w*`
	`\bGlacier\b`
	`Intelligent[- ]Tiering`
	`\bscale to zero\b`
	`\bserverless\b`
	`\bauto[- ]?scal\w+`

Finding says	Nothing in the SOW addresses using no more resource than needed, which the Sustainability pillar asks for and which usually also lowers cost.
Suggested fix	Mention Graviton or managed serverless where it fits, storage lifecycle policies, and switching off non-production outside working hours.
Evidence shown	The SOW contains no wording on this topic.
Worked examples (executed)	**FIRES** A draft that says "Nothing has been decided yet." **DOES NOT FIRE** Graviton (arm64) throughout and S3 Intelligent-Tiering lifecycle policies.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

4. Contradiction rules (what the SOW says is wrong)

These read what the SOW states. Their finding text below is the real output of the review on a triggering draft.

wa_operations_compute_decision_open	Operational Excellence | **HIGH** | contradiction rule
Finding title	**Competing compute options left undecided**
Fires when	Within **one sentence or bullet**: two or more of the compute options (Amazon ECS, Amazon EKS, AWS Fargate, Amazon EC2, AWS Lambda) are named **and** the sentence has an undecided marker (or, versus, vs, either, one of, any of, TBD, to be determined/decided/confirmed, option(s), could be, may be, as appropriate/needed/required). Sentences describing one combined choice are excluded, such as an ECS service on Fargate, or ECS with EC2.

Detects (regex)	`\bor\b`
	`\bversus\b`
	`\bvs\.?\b`
	`\beither\b`
	`\bone of\b`
	`\bany of\b`
	`\bTBD\b`
	`to be (?:determined|decided|confirmed|agreed|selected)`
	`\boptions?\b`
	`\bcould be\b`
	`\bmay be\b`
	`\bas (?:appropriate|needed|required)\b`
	`excluded: \bECS\b[^.\n]{0,40}\b(?:on|with|using)\b[^.\n]{0,20}\bFargate\b`
	`excluded: \bEKS\b[^.\n]{0,40}\b(?:on|with|using)\b[^.\n]{0,20}\bFargate\b`
	`excluded: \bFargate\b[^.\n]{0,30}\b(?:on|for)\b[^.\n]{0,20}\b(?:ECS|EKS)\b`
	`excluded: \bECS\b[^.\n]{0,40}\b(?:on|with|using)\b[^.\n]{0,20}\bEC2\b`

Finding says	The SOW offers AWS Fargate / Amazon ECS / Amazon EKS as alternatives rather than recording which one is chosen and why. The architecture diagram and the cost estimate are both built from this choice, so leaving it open forces a loop back through all three stages.
Suggested fix	Pick one compute model in the SOW, state the reason, and record the others as rejected alternatives or out of scope.
Evidence shown	- Container platform using Amazon ECS, Amazon EKS or AWS Fargate.
Worked examples (executed)	**FIRES** Container platform using Amazon ECS, Amazon EKS or AWS Fargate. **FIRES** Compute platform: Amazon EKS or Amazon EC2, TBD during design. **DOES NOT FIRE** An Amazon ECS service on AWS Fargate (ARM64, 2 tasks) serves the API. **DOES NOT FIRE** AWS Lambda handles ingestion events. An Amazon ECS service on AWS Fargate serves the API. **DOES NOT FIRE** We will use AWS Fargate for all services. Amazon EKS is out of scope.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

wa_reliability_single_az	Reliability | **HIGH** | contradiction rule
Finding title	**Workload scoped to a single Availability Zone**
Fires when	The SOW text matches a single-AZ pattern, **or** an upstream architecture assumption or NFR does.

Detects (regex)	`single[- ]A\.?Z`
	`\bone Availability Zone\b`
	`a single Availability Zone`
	`availability[_ ]zones?\s*[:=]\s*1\b`
	`\bmulti[_ ]az\s*[:=]\s*(?:false|no)\b`

Finding says	The Reliability pillar expects a workload to span at least two Availability Zones (REL10-BP02). A single zone means a zone failure is an outage recoverable only from backup, and because the diagram feeds the cost calculator the quoted price is a single-zone price.
Suggested fix	Scope at least two Availability Zones, or state plainly that single-AZ is a deliberate pilot limitation with the residual risk and the path to multi-AZ.
Evidence shown	- The pilot is deployed in a single Availability Zone.

Worked examples (executed)	**FIRES** The pilot is deployed in a single Availability Zone. **DOES NOT FIRE** The solution runs across two Availability Zones with Multi-AZ RDS.
AWS reference	REL10-BP01: Deploy the workload to multiple locations https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_fault_isolation_multiaz_region_system REL10-BP02: Select the appropriate locations for your multi-location deployment https://docs.aws.amazon.com/wellarchitected/2023-10-03/framework/rel_fault_isolation_select_location.html

wa_reliability_overclaimed_resiliency	Reliability | **HIGH** | contradiction rule
Finding title	**Resiliency claim the scope does not support**
Fires when	The SOW makes a well-architected, high-availability or fault-tolerance claim **and** single-AZ evidence (previous rule) is present.

Detects (regex)	`well[- ]architected`
	`\bhighly available\b`
	`\bhigh availability\b`
	`\bfault[- ]toleran\w+`
	`\bresilient by design\b`

Finding says	The SOW describes the solution as well-architected or highly available while scoping a single Availability Zone. The claim is not defensible in a customer review and undermines the rest of the document.
Suggested fix	Either scope two or more Availability Zones, or remove the resiliency claim and describe the single-AZ limitation honestly.
Evidence shown	- A well-architected, highly available platform in a single Availability Zone.
Worked examples (executed)	**FIRES** A well-architected, highly available platform in a single Availability Zone. **DOES NOT FIRE** A well-architected, highly available platform across two Availability Zones.
AWS reference	REL10-BP02: Select the appropriate locations for your multi-location deployment https://docs.aws.amazon.com/wellarchitected/2023-10-03/framework/rel_fault_isolation_select_location.html

wa_operations_missing_out_of_scope	Operational Excellence | **HIGH** | contradiction rule
Finding title	**No out-of-scope list**
Fires when	None of these phrases appears: "out of scope", "out-of-scope", "exclusion(s)", "not included". The architecture stage takes the in/out split as its input, so a SOW without one cannot constrain the design.

Detects (regex)	`out of scope`
	`out-of-scope`
	`exclusions?`
	`not included`

Finding says	The SOW states what is in scope but never what is excluded. The architecture stage takes this split as its input, so without it the design has nothing stopping it from adding services the customer never agreed to pay for.
Suggested fix	Add an explicit Out of Scope list naming the capabilities and services excluded from this engagement.
Evidence shown	No 'out of scope', 'exclusions' or 'not included' wording found in the draft.
Worked examples (executed)	**FIRES** A SOW with an In Scope list and no exclusions. **DOES NOT FIRE** The same SOW with an "Out of Scope", "Exclusions" or "Not included" section.
AWS reference	Pillar-level mapping only. No specific best practice is cited for this rule.

5. Optional model review

A Claude reviewer can read the SOW and the intake data for what the rules cannot judge: whether a chosen service fits the stated requirements, a requirement the scope does not cover, a commitment the architecture cannot keep. It is instructed to ignore anything the rules already cover, to ignore pricing, to cite evidence for every finding, and to return at most 12 findings.

Its findings are **advisory** and merge behind the rules, keyed on (pillar, title). On a collision the rule's finding wins, because it carries the matched text. A model finding of high severity does close the gate.

State today: off in the deployed pipeline (`WaVerify=false`). The SOW API route accepts `verify: true` per request, which costs one model call.

6. Availability rules applied when the architecture is drawn

The review above catches a single-AZ scope in the SOW. The architecture stage then designs against a configured availability model, so the diagram, the usage figures and the price agree. The setting is `AVAILABILITY_ZONES`; the default is 2, the baseline for the Reliability pillar (REL10-BP02: deploy to multiple Availability Zones where possible). The text below is the instruction the design model receives.

With AVAILABILITY_ZONES=2

```
## Availability model (authoritative)
- **One Region, two Availability Zones.** Use the Region the SRS states (default us-east-1) and name exactly 2 AZs in `availability_zones` (default `<region>a`, `<region>b`, ... in order).
- Spread every component that is not Region-level or global across the 2 zones, and say so in its configuration. Subnets come in one public and one private per zone, each its own boundary with the AZ boundary as `parent`.
- Managed services that offer it run with their multi-AZ option on: Amazon RDS and Amazon ElastiCache use a Multi-AZ deployment with a standby in another zone, Application Load Balancers and NAT Gateways exist in each of the 2 zones, and Auto Scaling groups and ECS or EKS services spread their tasks across all of them.
- State the multi-AZ choice in the `sku_summary` of every component it applies to (for example "db.t4g.medium, PostgreSQL 17, gp3 50 GiB, Multi-AZ"), because the pricing analyst reads that line.
- Cross-AZ data transfer and the per-zone NAT Gateways are real costs. Do not hide them: a two Availability Zones deployment is more expensive than one zone, and that is the intended trade-off for resiliency.
- Never place the whole workload in a single AZ, and never add a second Region or a DR Region unless the SRS asks for one.
```

With AVAILABILITY_ZONES=1

```
## Availability model (authoritative)
- **One Region and one Availability Zone.** Use the Region the SRS states (default us-east-1) and name exactly one AZ in `availability_zones` (default `<region>a`). Never use Multi-AZ deployment options, standby instances, cross-AZ replicas, workloads in a second AZ or a DR Region. Recovery comes from backups and snapshots.
- This single-AZ scope is a deliberate exception to the Well-Architected Reliability pillar. For every component whose availability depends on it, record the residual risk and the backup-based recovery path in the decision's consequences, and raise an open question about expanding to two AZs after the pilot.
- Do not describe the solution as highly available or as following Well-Architected Reliability best practice.
```

The usage figures that feed the cost calculator (`multi_az`, `availability_zones`) are set in code from the finished design, never taken from the model, so the price follows the drawn diagram.

7. What this does not do

Limit	Consequence
A keyword is a commitment.	A rule is satisfied by any matching word. This sentence satisfies the backup rule, verified when this document was built: `We do not provide any backups.`. The model review (off by default) is the only layer that reads for meaning.
Not the AWS service.	It does not call the AWS Well-Architected Tool or the AWS Well-Architected Agent. The AWS agent's on-demand review takes CDK or Terraform from an S3 URI, not prose, so it cannot review a SOW.
Our checklist, hand-written.	The rules are fixed in code. They do not update when AWS updates the Framework; changing one means a code change and a redeploy of the SOW agent.
Not every pillar equally.	Six pillars are covered, but Performance Efficiency, Cost Optimization and Sustainability have one low-depth rule each.
The SOW only.	The diagram and the cost estimate are not Well-Architected reviewed. The final review (Section 8) checks consistency, not pillars.
References are partial.	Only the best practices listed in the rule cards were checked against AWS documentation. Other rules map to a pillar only.

8. The final combined review (stage 5)

After the architecture and cost exist, the SOW agent checks the finished SOW with its compliance rules, plus an optional model reviewer. These are consistency checks, not Well-Architected checks. A failed **required** rule stops the run; an advisory one is reported only.

Rule	Severity	What it checks
scope-section-present Scope section present	required	Every SOW must carry a Scope section defining in-scope work.
pricing-section-present Pricing section present	required	A Pricing / Commercials section must state the engagement price.
pricing-refers-to-cost-estimate Pricing refers to the cost estimate	required	The pricing section must carry the cost calculator link and must not restate or tabulate AWS run-rate figures. The cost estimate agent's calculator is the single source of truth; duplicating its numbers in the SOW lets the two drift apart.
services-covered Architecture services covered in scope	advisory	Every architecture service should appear in scope or be explicitly out of scope.
timeline-consistent SOW timeline matches intake	advisory	The schedule in the SOW must be consistent with the intake timeline.

9. Evidence and provenance

Tests. 29 passed in `tests/test_wa_review.py`, run when this file was built. They cover each rule firing and not firing, ordering, the gate, the assessment profile, and the model-pass merge. A separate run of the deployed pipeline on a SOW that passed the review ended with real cost-estimate link, diagram and .docx.

Source files this document was generated from (SHA-256, so you can tell if the code has moved on):

File	SHA-256
sow-generator-agent/deployment/agent/sow_generator/services/wa_review/checks.py	`26f0a4f436beb2ffe813d48bd644ccabaa0a871d6726682ae0668de9b1bca4cc`
sow-generator-agent/deployment/agent/sow_generator/services/wa_review/reviewer.py	`94d455091cc03b6521e81befe9ab5bbdbf09eac2d27111a55aa6dc40ea7cb475`
sow-generator-agent/deployment/agent/sow_generator/schemas/wa.py	`6be326d3824a289f8772ad3ce1b88eeca06a812b1aa16f8a049c860f4ea57dc2`
sow-generator-agent/deployment/agent/sow_generator/resources/prompts/wa_review.txt	`83484b810029d66d12a6c3c1ca714903667400c27e12febf02eb88cc0594b6eb`
sow-generator-agent/deployment/agent/sow_generator/resources/policies/sow_compliance_rules.yaml	`2486084f1ad572e6a58c4ecf45fb316ad087b9730ddf992daccf3963e2727760`
architecture-agent/deployment/agent/adr_generator/availability.py	`a3c50a2fb4c50677a024f1e8a7b92182d6e3a2a7a1061f5967621df773ccf4e1`
pipeline-orchestrator/deployment/api/orchestrator/pipeline.py	`82ca0935e81622936b996dbc818699cd48e076a9e7707a01f9643a590bc8dd2c`

AWS documentation cited (best-practice ids and titles checked against docs.aws.amazon.com):

Id	Title and link
REL09-BP01	Identify and back up all data that needs to be backed up, or reproduce the data from sources https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_backing_up_data_identified_backups_data.html
REL10-BP01	Deploy the workload to multiple locations https://docs.aws.amazon.com/wellarchitected/latest/framework/rel_fault_isolation_multiaz_region_system
REL10-BP02	Select the appropriate locations for your multi-location deployment https://docs.aws.amazon.com/wellarchitected/2023-10-03/framework/rel_fault_isolation_select_location.html
SEC03-BP02	Grant least privilege access https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_permissions_least_privileges.html
SEC08-BP02	Enforce encryption at rest https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_protect_data_rest_encrypt.html
SEC09-BP02	Enforce encryption in transit https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/sec_protect_data_transit_encrypt.html
OPS08-BP01	Analyze workload metrics (workload observability) https://docs.aws.amazon.com/wellarchitected/latest/framework/ops_workload_observability_analyze_workload_metrics.html

To regenerate: `python3 pipeline-orchestrator/docs/build_waf_rules_pdf.py`. The build fails if any example above stops behaving as stated.